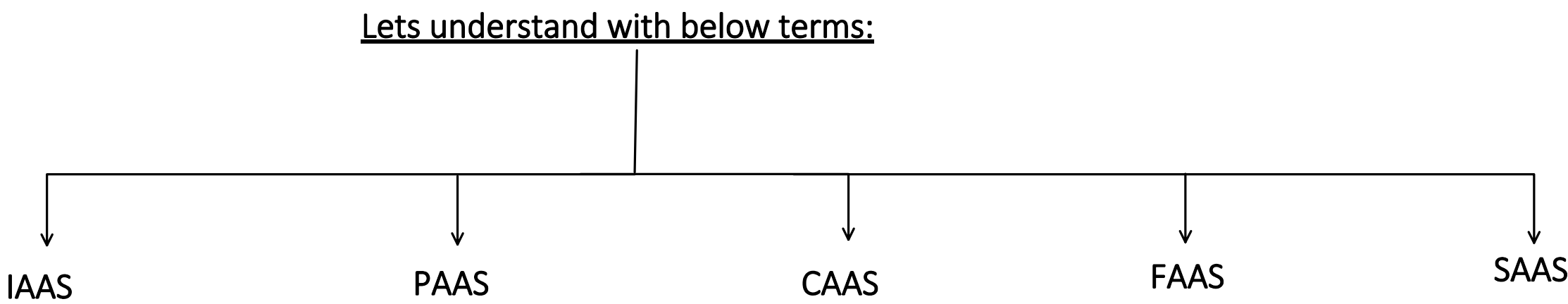


AWS: provides reliable, scalable, on-demand cloud computing services.



To run 1 application, below are the most common things we required:

Application	Ex: spring boot code (app.jar)
Data	Like MySQL, S3 etc.
Runtime	Like JVM (which execute the bytecode)
Middleware	Like Tomcat which sits between application and OS. - Request routing - HTTP handling - Connection management etc.
OS	Like Ubuntu, windows etc.
Virtualization	1 physical server - 1application is costly 1 physical server - Many Virtual machines
RAM	In memory
CPU	computing
Storage	HDD, SSD (permanent storages)
Networking	Routers, switches, cables etc.

IAAS (Infrastructure as a Service):

- Infrastructure is managed by cloud provider.
- Software and application is managed by us.

These comes under Infrastructure:

- Virtualization
- Server (CPU/RAM)
- Storage
- Networking

Application	Ex: spring boot code (app.jar)
Data	Like MySQL, S3 etc.
Runtime	Like JVM (which execute the bytecode)
Middleware	Like Tomcat which sits between application and OS. - Request routing - HTTP handling - Connection management etc.
OS	Like Ubuntu, windows etc.
Virtualization	1 physical server - 1application is costly 1 physical server - Many Virtual machines
RAM	In memory
CPU	computing
Storage	HDD, SSD (permanent storages)
Networking	Routers, switches, cables etc.

Example: EC2

- After we launch EC2

We have to:

- Install Ubuntu
- Install Java
- Install MySQL
- Setup load balancer
- Deploy application
- etc.

So, we get full control over the environment.

- JVM tuning
- Custom tomcat configs etc.

PAAS (Platform as a Service):

- We only need to think about our application, business logic, our application data.
- Rest is all managed by cloud provider.

Application	Ex: spring boot code (app.jar)
Data	Like MySQL, S3 etc.
Runtime	Like JVM (which execute the bytecode)
Middleware	Like Tomcat, which sits between application and OS. - Request routing - HTTP handling - Connection management etc.

These comes under Platform:

- Runtime
- Middleware
- OS
- Virtualization
- Server (CPU/RAM)
- Storage
- Networking

OS	Like Ubuntu, windows etc.
Virtualization	1 physical server - 1application is costly 1 physical server - Many Virtual machines
RAM	In memory
CPU	computing
Storage	HDD, SSD (permanent storages)
Networking	Routers, switches, cables etc.

Example: AWS Elastic Beanstalk

All we have to do is upload 'MyApp.jar', internally it will create everything for us:

- EC2
- OS
- Java runtime
- Tomcat
- Load balancer
- security
- Scaling
- Etc.

Example: AWS Elastic Beanstalk

Used for simplified application development. Especially for small scale system.

As it abstract lot of configurations from developer, Large scale enterprises often prefer EC2 or Container platform(like ECS, Kubernetes) for greater infrastructure control and scalability.

One major problem for developer is:
It works on my machine but not in production.

This happens because development and production environment is different.

For example:
During development, we are using Java 17 but in production Java 21 is used.

There can be many other reasons like:

- Missing dependencies
- Different library versions
- Different environment variables
- Different file paths

Then comes the CONTAINER.

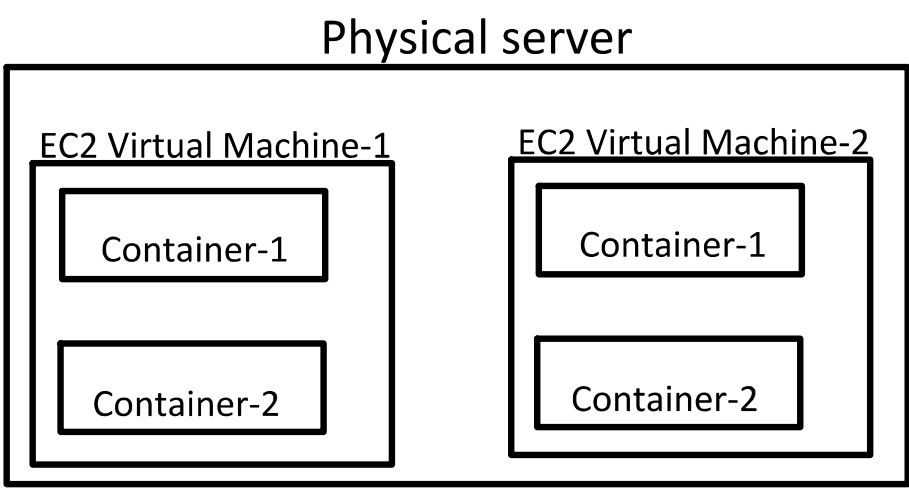
Container packed our:

- Application
- Runtime
- Dependencies

CAAS (Container as a Service):

Give this container to Cloud provider, it will run and manage for us.

Popular CAAS technology: Kubernetes (considered it as giant Container Orchestration Engine), AWS ECS



In CAAS:

Our responsibility:

- Provide docker image
- Tell configuration:
 - How many containers we needed
 - CPU/RAM limit
 - Scaling rules etc.

CAAS provider responsibility:

- Which VM should run the container
- Restarting failed containers
- Scaling
- Load balancing
- Etc.

FAAS (Function as a Service):

All we have to do is write a function. Platform execute it when required.

ex: AWS Lambda

And rest is all managed by provider.

Servers

- Runtime
- Scaling

- Infrastructure
- Availability
- Execution environment

Use it when, we have tasks which are short lived and event driven.

For example:

Video conversion from 4K to 1080p, 720p.

- Now we have written the function and uploaded to AWS Lambda. When video is uploaded in 4k in S3, then it will publish an event, which will invoke our Lambda function. It will convert the video in multiple file formats , save it back to S3.

- No video uploads for 5 hours then, our function is not consuming any compute resources.

But if suddenly:

- 1000 videos uploaded, AWS automatically scales function execution.

SAAS (Software as a Service):

Complete software is managed by Provide, we only pay for its usage.

tools like:

- Chime
(used for video meetings, chat etc.)
- Amazon Workmail
- Amazon QuickSight for business dashboards.

Application	Ex: spring boot code (app.jar)
Data	Like MySQL, S3 etc.
Runtime	Like JVM (which execute the bytecode)
Middleware	Like Tomcat, which sits between application and OS. • Request routing • HTTP handling • Connection management etc.
OS	Like Ubuntu, windows etc.
Virtualization	1 physical server - 1application is costly 1 physical server - Many Virtual machines
RAM	In memory
CPU	computing
Storage	HDD, SSD (permanent storages)
Networking	Routers, switches, cables etc.